

05_direct_unk_diagnostic

September 21, 2026

1 Direct <unk> Diagnostic

This notebook investigates why the completed **SeamlessM4T v2-large Direct pipeline** produced literal <unk> tokens in some Simplified Chinese translations.

It does **not** modify the official Direct predictions and does **not** rerun speech translation.

It checks:

1. how many outputs contain <unk>;
2. whether the rate differs by accent group;
3. whether <unk> is associated with source length, audio duration, runtime, or generated-token count;
4. whether repeated source sentences show the same <unk> behaviour across speakers;
5. whether the SeamlessM4T tokenizer maps text in the Chinese reference translations to the actual unknown-token ID;
6. representative cases for manual inspection.

The official 4,200-sample run remains the baseline.

1.1 1. Install and import libraries

Only the SeamlessM4T processor/tokenizer is loaded. The large neural model is **not** required.

```
[1]: %pip install -q "transformers==4.57.6" sentencepiece pandas numpy matplotlib
      ↪huggingface_hub ipywidgets protobuf sentencepiece
```

WARNING: You are using pip version 21.2.4; however, version 26.0.1 is available.

You should consider upgrading via the

'/Users/nlh/Downloads/ua_lectures/CS760/project/cs760-accent-st-robustness-ml-research/.venv/bin/python -m pip install --upgrade pip' command.

Note: you may need to restart the kernel to use updated packages.

```
[2]: from pathlib import Path
      import json
      import re
      import time
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import transformers
from transformers import AutoProcessor

print("Transformers:", transformers.__version__)

```

/Users/nlh/Downloads/uoa_lectures/CS760/project/cs760-accent-st-robustness-ml-research/.venv/lib/python3.9/site-packages/urllib3/__init__.py:35:

NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See:

<https://github.com/urllib3/urllib3/issues/3020>

warnings.warn(

None of PyTorch, TensorFlow >= 2.0, or Flax have been found. Models won't be available and only tokenizers, configuration and file/data utilities can be used.

None of PyTorch, TensorFlow >= 2.0, or Flax have been found. Models won't be available and only tokenizers, configuration and file/data utilities can be used.

Transformers: 4.57.6

1.2 2. Resolve project paths

The code supports either `direct_full_run_1788151795` or `direct_run_1788151795`. Edit the candidate paths if your local repository uses another name.

```

[3]: def find_project_root():
    cwd = Path.cwd().resolve()
    for candidate in [cwd, cwd.parent]:
        if (candidate / "data").exists() and (candidate / "runs").exists():
            return candidate
    return cwd

PROJECT_ROOT = find_project_root()

prediction_candidates = [
    PROJECT_ROOT / "runs/direct_full_run_1788151795/direct_predictions.csv",
    PROJECT_ROOT / "runs/direct_run_1788151795/direct_predictions.csv",
]

runtime_candidates = [
    PROJECT_ROOT / "runs/direct_full_run_1788151795/direct_runtime.csv",
    PROJECT_ROOT / "runs/direct_run_1788151795/direct_runtime.csv",
]

DIRECT_PREDICTIONS_PATH = next(

```

```

    (p for p in prediction_candidates if p.is_file()),
    prediction_candidates[0],
)

DIRECT_RUNTIME_PATH = next(
    (p for p in runtime_candidates if p.is_file()),
    runtime_candidates[0],
)

MODEL_ID = "facebook/seamless-m4t-v2-large"
MODEL_REVISION = "5f8cc790b19fc3f67a61c105133b20b34e3dcb76"

OUTPUT_DIR = PROJECT_ROOT / "runs" / f"direct_unk_diagnostic_{int(time.time())}"
OUTPUT_DIR.mkdir(parents=True, exist_ok=False)

print("Project root:", PROJECT_ROOT)
print("Predictions:", DIRECT_PREDICTIONS_PATH)
print("Runtime:", DIRECT_RUNTIME_PATH)
print("Output:", OUTPUT_DIR)

if not DIRECT_PREDICTIONS_PATH.is_file():
    raise FileNotFoundError(
        f"Direct predictions not found: {DIRECT_PREDICTIONS_PATH}"
    )

```

```

Project root: /Users/nlh/Downloads/uoa_lectures/CS760/project/cs760-accent-st-
robustness-ml-research
Predictions: /Users/nlh/Downloads/uoa_lectures/CS760/project/cs760-accent-st-
robustness-ml-research/runs/direct_full_run_1788151795/direct_predictions.csv
Runtime: /Users/nlh/Downloads/uoa_lectures/CS760/project/cs760-accent-st-
robustness-ml-research/runs/direct_full_run_1788151795/direct_runtime.csv
Output: /Users/nlh/Downloads/uoa_lectures/CS760/project/cs760-accent-st-
robustness-ml-research/runs/direct_unk_diagnostic_1789953533

```

1.3 3. Load completed Direct predictions

```

[4]: pred = pd.read_csv(DIRECT_PREDICTIONS_PATH)

required = {
    "sentence",
    "sentence_id",
    "primary_accent",
    "sample_id",
    "reference_translation_zh",
    "direct_translation",
}

```

```

missing = required - set(pred.columns)
if missing:
    raise KeyError(f"Missing required columns: {sorted(missing)}")

if len(pred) != 4200:
    raise ValueError(f"Expected 4,200 rows, found {len(pred):,}")

pred = pred.copy()
pred["row_index"] = np.arange(len(pred))

pred["direct_has_unk"] = (
    pred["direct_translation"]
    .fillna("")
    .astype(str)
    .str.contains("<unk>", regex=False)
)

pred["direct_unk_count"] = (
    pred["direct_translation"]
    .fillna("")
    .astype(str)
    .str.count(re.escape("<unk>"))
)

print("Rows:", len(pred))
print("Rows containing <unk>:", int(pred["direct_has_unk"].sum()))
print("Rate:", f'{100 * pred["direct_has_unk"].mean():.2f}%')
print("Total literal <unk> occurrences:", int(pred["direct_unk_count"].sum()))

```

```

Rows: 4200
Rows containing <unk>: 508
Rate: 12.10%
Total literal <unk> occurrences: 599

```

1.4 4. Distribution of <unk> counts per output

```

[5]: unk_count_distribution = (
    pred["direct_unk_count"]
    .value_counts()
    .sort_index()
    .rename_axis("unk_tokens_in_output")
    .reset_index(name="clips")
)

display(unk_count_distribution)

unk_count_distribution.to_csv(

```

```

OUTPUT_DIR / "unk_count_distribution.csv",
index=False,
)

```

	unk_tokens_in_output	clips
0	0	3692
1	1	434
2	2	60
3	3	11
4	4	3

1.5 5. Descriptive <unk> rate by accent

This is descriptive only. It does not establish an accent effect because groups differ in speakers, sentence content, and recordings.

```

[6]: accent_summary = (
    pred.groupby("primary_accent", dropna=False)
    .agg(
        clips=("direct_has_unk", "size"),
        unk_clips=("direct_has_unk", "sum"),
        total_unk_tokens=("direct_unk_count", "sum"),
    )
    .reset_index()
)

accent_summary["unk_clip_rate_pct"] = (
    100 * accent_summary["unk_clips"] / accent_summary["clips"]
)

accent_summary = accent_summary.sort_values(
    "unk_clip_rate_pct",
    ascending=False,
)

display(accent_summary)

accent_summary.to_csv(
    OUTPUT_DIR / "accent_unk_summary.csv",
    index=False,
)

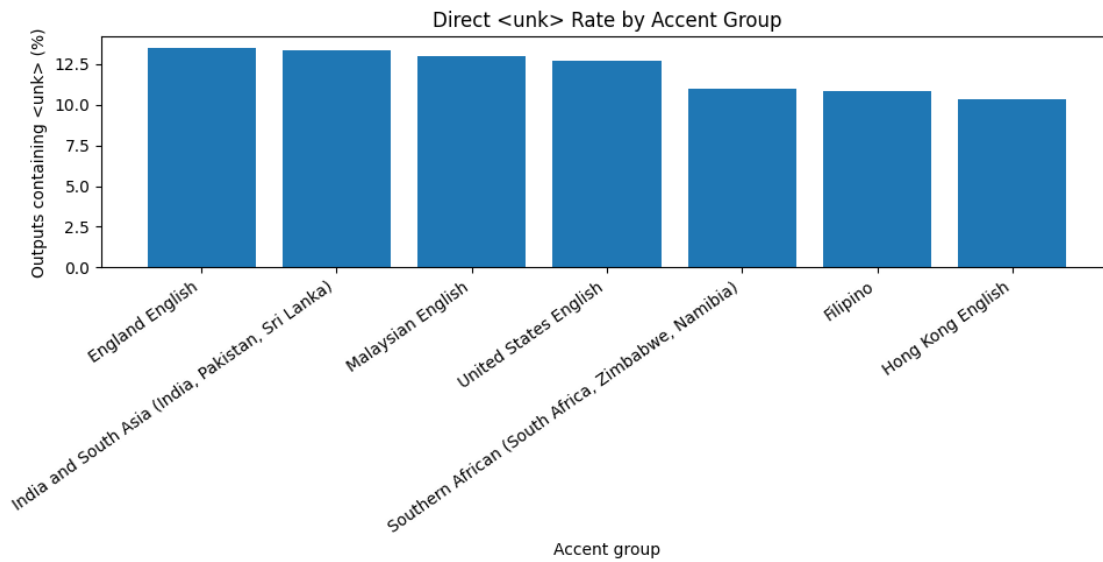
plt.figure(figsize=(10, 5))
plt.bar(
    accent_summary["primary_accent"],
    accent_summary["unk_clip_rate_pct"],
)
plt.ylabel("Outputs containing <unk> (%)")

```

```
plt.xlabel("Accent group")
plt.title("Direct <unk> Rate by Accent Group")
plt.xticks(rotation=35, ha="right")
plt.tight_layout()
plt.show()
```

	primary_accent	clips	unk_clips	\
0	England English	600	81	
3	India and South Asia (India, Pakistan, Sri Lanka)	600	80	
4	Malaysian English	600	78	
6	United States English	600	76	
5	Southern African (South Africa, Zimbabwe, Nami...	600	66	
1	Filipino	600	65	
2	Hong Kong English	600	62	

	total_unk_tokens	unk_clip_rate_pct
0	99	13.500000
3	99	13.333333
4	94	13.000000
6	83	12.666667
5	79	11.000000
1	74	10.833333
2	71	10.333333



1.6 6. Join runtime diagnostics

The earlier runtime file is aligned using `sample_index`, which corresponds to the row order of the official Direct run.

```

[7]: if DIRECT_RUNTIME_PATH.is_file():
    runtime = pd.read_csv(DIRECT_RUNTIME_PATH).copy()

    if "sample_index" not in runtime.columns:
        raise KeyError("Runtime CSV does not contain sample_index.")

    runtime["sample_index"] = pd.to_numeric(
        runtime["sample_index"],
        errors="coerce",
    ).astype("Int64")

    if runtime["sample_index"].isna().any():
        raise ValueError("Runtime CSV contains invalid sample_index values.")

    if runtime["sample_index"].duplicated().any():
        raise ValueError("Runtime CSV contains duplicate sample_index values.")

    keep = [
        c for c in [
            "sample_index",
            "audio_duration_sec",
            "direct_runtime_sec",
            "direct_generation_runtime_sec",
            "direct_realtime_factor",
            "generated_token_count",
            "generation_limit_reached",
            "direct_status",
        ]
        if c in runtime.columns
    ]

    pred = pred.merge(
        runtime[keep],
        left_on="row_index",
        right_on="sample_index",
        how="left",
        validate="one_to_one",
    )

    print("Runtime diagnostics joined.")
else:
    print("Runtime CSV not found. Runtime analysis will be skipped.")

```

Runtime diagnostics joined.

```

[8]: pred["source_word_count"] = (
    pred["sentence"]
    .fillna("")

```

```

        .astype(str)
        .str.split()
        .str.len()
    )

    candidate_measures = [
        "source_word_count",
        "audio_duration_sec",
        "direct_runtime_sec",
        "direct_generation_runtime_sec",
        "generated_token_count",
    ]

    measures = [c for c in candidate_measures if c in pred.columns]

    rows = []

    for col in measures:
        values = pd.to_numeric(pred[col], errors="coerce")

        for flag, label in [(True, "contains_<unk>"), (False, "no_<unk>")]:
            subset = values[pred["direct_has_unk"] == flag].dropna()

            rows.append({
                "measure": col,
                "group": label,
                "n": len(subset),
                "mean": subset.mean(),
                "median": subset.median(),
                "std": subset.std(),
                "min": subset.min(),
                "max": subset.max(),
            })

    numeric_summary = pd.DataFrame(rows)

    display(numeric_summary)

    numeric_summary.to_csv(
        OUTPUT_DIR / "unk_vs_nonunk_numeric_summary.csv",
        index=False,
    )

```

	measure	group	n	mean	median \
0	source_word_count	contains_<unk>	508	9.881890	10.000000
1	source_word_count	no_<unk>	3692	8.862134	8.000000
2	audio_duration_sec	contains_<unk>	508	5.047417	4.860000
3	audio_duration_sec	no_<unk>	3692	4.560501	4.344000

4	direct_runtime_sec	contains_<unk>	508	1.052853	0.985303
5	direct_runtime_sec	no_<unk>	3692	0.911083	0.855187
6	direct_generation_runtime_sec	contains_<unk>	508	1.020577	0.953449
7	direct_generation_runtime_sec	no_<unk>	3692	0.880881	0.826161
8	generated_token_count	contains_<unk>	508	19.370079	19.000000
9	generated_token_count	no_<unk>	3692	14.325840	14.000000

	std	min	max
0	3.511299	2.000000	21.000000
1	3.592295	2.000000	26.000000
2	1.696485	1.824000	9.360000
3	1.662400	1.728000	9.384000
4	0.270867	0.664371	2.420101
5	0.234716	0.542248	2.144140
6	0.262306	0.643862	2.351985
7	0.226506	0.525718	2.069366
8	5.153353	9.000000	42.000000
9	4.770513	5.000000	36.000000

1.7 7. Repeated-sentence consistency

If the same source sentence is spoken by different speakers and repeatedly produces <unk>, this supports a content/model-related pattern more than a purely speaker-specific explanation.

```
[9]: repeated = (
    pred.groupby("sentence_id", dropna=False)
    .agg(
        occurrences=("sentence_id", "size"),
        speakers=("sample_id", "nunique"),
        accents=("primary_accent", "nunique"),
        unk_occurrences=("direct_has_unk", "sum"),
        sentence=("sentence", "first"),
    )
    .reset_index()
)

repeated = repeated[repeated["occurrences"] > 1].copy()

repeated["unk_rate"] = (
    repeated["unk_occurrences"] / repeated["occurrences"]
)

repeated["pattern"] = np.select(
    [
        repeated["unk_rate"].eq(0),
        repeated["unk_rate"].eq(1),
    ],
    [
```

```

        "never_unk",
        "always_unk",
    ],
    default="mixed",
)

pattern_counts = (
    repeated["pattern"]
    .value_counts()
    .rename_axis("pattern")
    .reset_index(name="repeated_sentences")
)

display(pattern_counts)

display(
    repeated[repeated["unk_occurrences"] > 0]
    .sort_values(["unk_rate", "occurrences"], ascending=[False, False])
    [
        [
            "sentence",
            "occurrences",
            "speakers",
            "accents",
            "unk_occurrences",
            "unk_rate",
            "pattern",
        ]
    ]
    .head(30)
)

repeated.to_csv(
    OUTPUT_DIR / "repeated_sentence_unk_summary.csv",
    index=False,
)

```

```

      pattern  repeated_sentences
0  never_unk                256
1  always_unk                43
2      mixed                 2

```

```

                                sentence  occurrences  \
3344  The crowd swayed a little as I elbowed my way ...      5
3832                                He disappeared into the tent.      4
215   Yeah, I had a bit of a poke at that and failed...      3
1023  He stood irresolute for a moment and then scra...      3
3110   He grew strawberries, so you would not like it.      3

```

3476	The Englishman prodded him, and the boy asked ...	3
3494	But that was simply because my eye was tired.	3
329	Gossips are frogs, they drink and talk.	2
781	Pull the hook to toggle the jukebox.	2
1660	I have blisters on my hands because I was boul...	2
1802	The diver forced open the oyster, revealing a ...	2
1815	Take shelter in this tent, but keep still.	2
1817	The hydrogen gas escaped.	2
1820	I bet you forgot the wine!	2
1924	He kept driving at full speed, totally unfazed...	2
1943	Oh, you are, eh?	2
1980	Nora finds herself ugly, because she has pimples.	2
2010	The boy mumbled an answer that allowed him to ...	2
2107	I ate a dish with rice, spaghetti, hummus, and...	2
2139	After twelve hours of rowing, Darius climbed o...	2
2310	She's a feisty little ferret.	2
2366	Canned pears lack full flavor.	2
2505	The grim reaper is waiting for you, reckless d...	2
2509	There were really, I think, two or three hundr...	2
2556	The penguin waddles across the ice, joining he...	2
2654	Another reason why Apple is superior to Micros...	2
2695	You will need some rubber boots.	2
2749	The thief wasn't there, and the owner brought ...	2
2919	Pitch the straw through the door of the stable.	2
2947	The turf and gravel around it seemed charred a...	2

	speakers	accents	unk_occurrences	unk_rate	pattern
3344	5	4	5	1.0	always_unk
3832	4	2	4	1.0	always_unk
215	3	3	3	1.0	always_unk
1023	3	2	3	1.0	always_unk
3110	3	3	3	1.0	always_unk
3476	3	3	3	1.0	always_unk
3494	3	3	3	1.0	always_unk
329	2	2	2	1.0	always_unk
781	2	2	2	1.0	always_unk
1660	2	2	2	1.0	always_unk
1802	2	2	2	1.0	always_unk
1815	2	2	2	1.0	always_unk
1817	2	2	2	1.0	always_unk
1820	2	2	2	1.0	always_unk
1924	2	2	2	1.0	always_unk
1943	2	1	2	1.0	always_unk
1980	2	1	2	1.0	always_unk
2010	2	2	2	1.0	always_unk
2107	2	2	2	1.0	always_unk
2139	2	2	2	1.0	always_unk
2310	2	1	2	1.0	always_unk

2366	2	2	2	1.0	always_unk
2505	2	2	2	1.0	always_unk
2509	2	2	2	1.0	always_unk
2556	2	2	2	1.0	always_unk
2654	2	2	2	1.0	always_unk
2695	2	1	2	1.0	always_unk
2749	2	1	2	1.0	always_unk
2919	2	2	2	1.0	always_unk
2947	2	2	2	1.0	always_unk

1.8 8. Load the exact SeamlessM4T processor/tokenizer

```
[17]: processor = AutoProcessor.from_pretrained(
        MODEL_ID,
        revision=MODEL_REVISION,
    )

tokenizer = processor.tokenizer

print("Tokenizer class:", tokenizer.__class__.__name__)
print("UNK token:", tokenizer.unk_token)
print("UNK token ID:", tokenizer.unk_token_id)

if tokenizer.unk_token_id is None:
    raise ValueError("Tokenizer does not expose an unk_token_id.")
```

```
Tokenizer class: SeamlessM4TTokenizerFast
UNK token: <unk>
UNK token ID: 1
```

1.9 9. Check tokenizer coverage of the human Chinese references

This is a key diagnostic.

If the **human reference translation itself** contains the tokenizer's unknown-token ID after encoding, that is direct evidence that the tokenizer cannot normally represent some target text.

If references encode normally but generated sequences still produce <unk>, the problem is more likely generation/decoding behaviour.

```
[19]: def inspect_reference(text):
        text = "" if pd.isna(text) else str(text)

        encoded = tokenizer(
            text,
            add_special_tokens=False,
        )

        ids = list(encoded.input_ids)
```

```

tokens = tokenizer.convert_ids_to_tokens(ids)

unk_count = sum(
    int(token_id == tokenizer.unk_token_id)
    for token_id in ids
)

return {
    "reference_token_count": len(ids),
    "reference_tokenizer_unk_count": unk_count,
    "reference_tokenizer_has_unk": bool(unk_count),
    "reference_tokens": tokens,
}

# Remove previously-created diagnostic columns if this cell is rerun.
diagnostic_columns = [
    "reference_token_count",
    "reference_tokenizer_unk_count",
    "reference_tokenizer_has_unk",
    "reference_tokens",
]

pred = pred.drop(
    columns=[
        col for col in diagnostic_columns
        if col in pred.columns
    ],
    errors="ignore",
)

# Run tokenizer inspection.
reference_info = pd.DataFrame(
    [
        inspect_reference(x)
        for x in pred["reference_translation_zh"]
    ],
    index=pred.index,
)

# Assign columns safely instead of concatenating.
for col in diagnostic_columns:
    pred[col] = reference_info[col]

```

```

# Sanity checks.
assert len(pred) == 4200
assert not pred.columns.duplicated().any()

print("Diagnostic columns added successfully.")

print(
    "Reference tokenizer contains UNK:",
    int(pred["reference_tokenizer_has_unk"].sum()),
    "/",
    len(pred),
)

print(
    "Direct output contains <unk>:",
    int(pred["direct_has_unk"].sum()),
    "/",
    len(pred),
)

# Cross-tabulation.
coverage = pd.crosstab(
    pred["reference_tokenizer_has_unk"],
    pred["direct_has_unk"],
    rownames=["Reference tokenizer contains UNK ID"],
    colnames=["Direct output contains literal <unk>"],
    margins=True,
)

display(coverage)

print(
    "Direct <unk> rows whose reference also tokenizes with UNK:",
    int(
        (
            pred["direct_has_unk"]
            & pred["reference_tokenizer_has_unk"]
        ).sum()
    ),
    "/",
    int(pred["direct_has_unk"].sum()),
)

```

```

Diagnostic columns added successfully.
Reference tokenizer contains UNK: 3498 / 4200
Direct output contains <unk>: 508 / 4200

```

Direct output contains literal <unk>	False	True	All
Reference tokenizer contains UNK ID			
False	642	60	702
True	3050	448	3498
All	3692	508	4200

Direct <unk> rows whose reference also tokenizes with UNK: 448 / 508

1.10 10. Inspect all affected rows

```
[20]: case_cols = [
        "row_index",
        "primary_accent",
        "sample_id",
        "sentence",
        "reference_translation_zh",
        "direct_translation",
        "direct_unk_count",
        "reference_tokenizer_unk_count",
        "reference_tokens",
    ]

    affected = pred.loc[pred["direct_has_unk"], case_cols].copy()

    display(affected.head(50))

    affected.to_csv(
        OUTPUT_DIR / "all_direct_unk_rows.csv",
        index=False,
        encoding="utf-8-sig",
    )
```

	row_index	primary_accent	sample_id	\
1	1	England English	spk_1385	
25	25	England English	spk_1516	
30	30	England English	spk_1122	
45	45	England English	spk_0383	
76	76	England English	spk_0859	
85	85	England English	spk_0996	
88	88	England English	spk_0010	
89	89	England English	spk_1183	
113	113	England English	spk_1149	
124	124	England English	spk_1097	
126	126	England English	spk_0695	
133	133	England English	spk_0516	
143	143	England English	spk_1347	
147	147	England English	spk_1221	
149	149	England English	spk_0597	

150	150	England	English	spk_1524
153	153	England	English	spk_0544
157	157	England	English	spk_1521
159	159	England	English	spk_0327
164	164	England	English	spk_0032
172	172	England	English	spk_1047
174	174	England	English	spk_0976
177	177	England	English	spk_0219
180	180	England	English	spk_1282
182	182	England	English	spk_1316
192	192	England	English	spk_0873
197	197	England	English	spk_1079
200	200	England	English	spk_0857
208	208	England	English	spk_0076
220	220	England	English	spk_1435
221	221	England	English	spk_1520
232	232	England	English	spk_0303
234	234	England	English	spk_0391
237	237	England	English	spk_0478
248	248	England	English	spk_0011
255	255	England	English	spk_0058
258	258	England	English	spk_0061
266	266	England	English	spk_1357
268	268	England	English	spk_0163
281	281	England	English	spk_0036
296	296	England	English	spk_0558
301	301	England	English	spk_1488
330	330	England	English	spk_0840
335	335	England	English	spk_0840
368	368	England	English	spk_1580
376	376	England	English	spk_0939
379	379	England	English	spk_0950
392	392	England	English	spk_1566
398	398	England	English	spk_1526
409	409	England	English	spk_1254

sentence \

1 However, after persistent calls, Hall-Jones re...

25 He stood irresolute for a moment and then scra...

30 The river is tidal when it passes through Darg...

45 The matter of the overall pennant was apparent...

76 Stark naked. Under bare poles. In puris natura...

85 Another reason why Apple is superior to Micros...

88 Zora, can you teach me how to yodel?

89 Restoration silver is characterized by embosse...

113 I have blisters on my hands because I was boul...

124 Where did you find that apple?

126 At her death she was the Dowager Countess of D...

133 The internal doors of the house would need to ...
 143 Chili powder blends are the primary flavor ing...
 147 I guess I used all the shells.
 149 There were really, I think, two or three hundr...
 150 Then the old man began to inspect the sheep, a...
 153 Some people like sea gulls, others don't.
 157 Boffer weapons are commonly used as props in l...
 159 Caution and adjustment of dosage as necessary ...
 164 Do you ever doodle things on a piece of paper ...
 172 A man of straw needs a woman of gold.
 174 You have enough gold to buy many sheep and man...
 177 the camel drivers exchanged information about ...
 180 Daily Telegraph journalist Toby Harnden descri...
 182 Keep your shirt on.
 192 Take shelter in this tent, but keep still.
 197 Football is often confused with soccer.
 200 And a whole cold chicken staring us right in t...
 208 The flag is red, blue, and yellow.
 220 Who is the blonde girl with the red skirt?
 221 Don't go off half-cocked.
 232 Yeah, I had a bit of a poke at that and failed...
 234 When he saw Henderson in his garden, he called...
 237 He also has a daughter, Amaiya Samuel.
 248 Without haste, yet without rest unhasting, yet...
 255 It's a homogeneous city.
 258 He noticed the unequal cooling of its surface.
 266 I like ice-cream.
 268 The school's mascot is the golden eagle and th...
 281 The hail pattered on the burnt brown grass.
 296 Ying told us to bring extra strawberries.
 301 He once lent his distinctive voice to an adver...
 330 It has also made watering holes inaccessible t...
 335 Like other royal insignia, the crown was not w...
 368 From the railway station in the distance came ...
 376 This is why oral rehydration salts include bot...
 379 I was hoping you might know a thing or two abo...
 392 The alchemist stopped abruptly, and bent to th...
 398 I ate a dish with rice, spaghetti, hummus, and...
 409 A steep trail is painful for our feet.

reference_translation_zh \

1 Hall-Jones

25

30

45

76

85

" "

88

Zora

89
 113
 124
 126
 133
 143
 147
 149
 150
 153
 157
 159
 164
 172
 174
 177
 180
 182
 192
 197
 200
 208
 220
 221
 232
 234
 237
 248
 255
 258
 266
 268
 281
 296
 301
 330
 335
 368
 376
 379
 392
 398
 409

Boffer

Football soccer

?

Henderson

Ribena

		direct_translation	direct_unk_count	\
1	, , -<unk>	, .	1	
25		, <unk> .		1
30		, <unk> .		1

45		<unk>	.	1	
76		, <unk>	, " "	1	
85	"<unk>	",	.	1	
88		, <unk>	?	1	
89		<unk>	.	1	
113		, <unk>	.	1	
124		<unk>	?	1	
126		, , <unk>	.	1	
133		, <unk>	.	1	
143	<unk>	<unk><unk><unk>	.	4	
147		<unk>	.	1	
149		, <unk>	.	1	
150		, <unk>	.	1	
153		<unk>	, .	1	
157		<unk>	.	1	
159	<unk>	<unk>	, .	2	
164		<unk>	?	1	
172		<unk>	.	1	
174		<unk>	.	1	
177		<unk>	, .	1	
180	" "	. <unk>	" <unk> "	2	
182		<unk>	.	1	
192		<unk>	, .	1	
197		<unk>	.	1	
200		<unk> <unk>	.	2	
208		<unk>	, .	1	
220		<unk>	?	1	
221		<unk>	.	1	
232		, <unk>	, .	1	
234		, <unk>	, .	1	
237		, . <unk>	.	1	
248		<unk>	, <unk>	, .	2
255		<unk>	.	1	
258		<unk>	.	1	
266		<unk>	.	1	
268		<unk>	, .	1	
281		<unk>	.	1	
296		<unk>	<unk>	.	2
301	<unk>	Ribena	.	1	
330		<unk>	.	1	
335		<unk>	, .	1	
368		, <unk>	<unk>	.	2
376		<unk>	.	1	
379		<unk>	?	1	
392		, <unk>	.	1	
398		, , , <unk>	.	1	
409		<unk>	.	1	

	reference_tokenizer_unk_count \
1	1
25	1
30	1
45	1
76	3
85	3
88	0
89	1
113	1
124	0
126	2
133	1
143	1
147	1
149	1
150	1
153	1
157	1
159	1
164	0
172	1
174	1
177	1
180	3
182	1
192	1
197	1
200	1
208	3
220	0
221	2
232	1
234	1
237	1
248	1
255	1
258	1
266	1
268	1
281	1
296	1
301	1
330	1
335	1
368	1
376	1
379	0

392 1
398 3
409 1

```

reference_tokens
1  [ , , , , , , , H, all, -, J, ones,...
25  [ , , , , , , , , , , , , , , , ...
30  [ , , , , , , , , , , , , , , <unk>]
45  [ , , , , , , , , , , , , , , <unk>]
76  [ , , , , <unk>, , , , , <unk>, , , ...
85  [ , <unk>, , , , , , , , , <unk>, ...
88  [ Z, ora, , , , , , , , , , , ?]
89  [ , , , , , , , , , , , , , , ...
113 [ , , , , , , , , , , , , , , ...
124 [ , , , , , , , , , ?]
126 [ , , , , , , , , , , <unk>, , , ...
133 [ , , , , , , , , , , , , , , <...
143 [ , , , , , , , , , , , , , , ...
147 [ , , , , , , , , , <unk>]
149 [ , , , , , , , , , , , , , , <unk>]
150 [ , , , , , , , , , , , , , , ...
153 [ , , , , , , , , , , , , , , <unk>, <pad>]
157 [ , , , Bo, ffer, , , , , , , , ...
159 [ , , , , , , , , , , , , , , , ...
164 [ , , , , , , , , , , , , , , ? , <pad>]
172 [ , , , , , , , , , , <unk>, <pad>]
174 [ , , , , , , , , , , , , , , <unk>]
177 [ , , , , , , , , , , , , , , , ...
180 [ , <unk>, , , , , <unk>, , , , , , ...
182 [ , , , , , , , , , <unk>, <pad>]
192 [ , , , , , , , , , , , , , , <unk>, <...
197 [ , , , Fo, otball, (, , , , , ), , s, o...
200 [ , , , , , , , , , , , , , , <unk>]
208 [ , , , , , <unk>, , <unk>, , , , , <unk>]
220 [ , , , , , , , , , , , , , , ?]
221 [ , , , , <unk>, , <unk>]
232 [ , , , , , , , , , , , , , , , ...
234 [ , , , , Henderson, , , , , , , , ...
237 [ , , , , , , , , , , , , , , , ...
248 [ , , , , , , , , , , , , , , , ...
255 [ , , , , , <unk>, <pad>]
258 [ , , , , , , , , , , , <unk>, <pad>]
266 [ , , , , , , , , , , <unk>, <pad>]
268 [ , , , , , , , , , , , , , , , ...
281 [ , , , , , , , , , , , , , , , <unk>]
296 [ , , , , , , , , , , , <unk>, <pad>]
301 [ , , , , , , , , , , , , , , , ...
330 [ , , , , , , , , , , , , , , <unk>, <...

```

```

335 [ , , , , , , , , , , , , , , ...
368 [ , , , , , , , , , , , , , , ...
376 [ , , , , , , , , , , , , , , ...
379 [ , , , , , , , , , , , , , , ...
392 [ , , , , , , , , , , , , , , <...
398 [ , , , , , , , <unk>, , , <unk>, , ...
409 [ , , , , , , , , , , , , <unk>, <pad>]

```

1.11 11. Optional manual Chinese tokenizer tests

Add suspicious Chinese words or characters here.

```

[21]: TEST_CHINESE_ITEMS = [
    " ",
    " ",
    " ",
    " ",
    " ",
    " ",
    " ",
    " "
]

manual_rows = []

for text in TEST_CHINESE_ITEMS:
    ids = tokenizer(
        text,
        add_special_tokens=False,
    ).input_ids

    tokens = tokenizer.convert_ids_to_tokens(ids)

    manual_rows.append({
        "text": text,
        "token_ids": ids,
        "tokens": tokens,
        "contains_unk_id": tokenizer.unk_token_id in ids,
        "unk_id_count": sum(
            int(token_id == tokenizer.unk_token_id)
            for token_id in ids
        ),
    })

manual_df = pd.DataFrame(manual_rows)
display(manual_df)

manual_df.to_csv(
    OUTPUT_DIR / "manual_tokenizer_tests.csv",

```

```

    index=False,
    encoding="utf-8-sig",
)

```

text	token_ids	tokens	contains_unk_id \
0	[247661, 197441]	[,]	False
1	[247661, 254833, 254338, 0]	[, , , <pad>]	False
2	[247661, 253098, 254312, 0]	[, , , <pad>]	False
3	[247661, 251771, 254362, 0]	[, , , <pad>]	False
4	[247661, 250865, 254341, 0]	[, , , <pad>]	False
5	[247661, 252872, 253836, 0]	[, , , <pad>]	False
6	[95723, 248725]	[,]	False

unk_id_count
0
1
2
3
4
5
6

1.12 12. Save diagnostic summary

```

[22]: summary = {
    "input_predictions": str(DIRECT_PREDICTIONS_PATH),
    "input_runtime": (
        str(DIRECT_RUNTIME_PATH)
        if DIRECT_RUNTIME_PATH.is_file()
        else None
    ),
    "model_id": MODEL_ID,
    "model_revision": MODEL_REVISION,
    "tokenizer_class": tokenizer.__class__.__name__,
    "unk_token": tokenizer.unk_token,
    "unk_token_id": tokenizer.unk_token_id,
    "total_rows": int(len(pred)),
    "direct_unk_rows": int(pred["direct_has_unk"].sum()),
    "direct_unk_row_rate": float(pred["direct_has_unk"].mean()),
    "total_literal_unk_occurrences": int(pred["direct_unk_count"].sum()),
    "reference_rows_tokenizing_to_unk": int(
        pred["reference_tokenizer_has_unk"].sum()
    ),
    "direct_unk_rows_whose_reference_tokenizes_to_unk": int(
        (
            pred["direct_has_unk"]
            & pred["reference_tokenizer_has_unk"]
        )
    )
}

```

```

        ).sum()
    ),
    "repeated_sentence_pattern_counts": {
        str(row["pattern"]): int(row["repeated_sentences"])
        for _, row in pattern_counts.iterrows()
    },
    "interpretation_note": (
        "These are diagnostic associations. They do not prove that a single "
        "factor causes SeamlessM4T to generate <unk>."
    ),
}

(OUTPUT_DIR / "diagnostic_summary.json").write_text(
    json.dumps(summary, indent=2, ensure_ascii=False),
    encoding="utf-8",
)

print(json.dumps(summary, indent=2, ensure_ascii=False))
print("\nSaved to:", OUTPUT_DIR)

```

```

{
    "input_predictions":
    "/Users/nlh/Downloads/uoa_lectures/CS760/project/cs760-accent-st-robustness-ml-
research/runs/direct_full_run_1788151795/direct_predictions.csv",
    "input_runtime":
    "/Users/nlh/Downloads/uoa_lectures/CS760/project/cs760-accent-st-robustness-ml-
research/runs/direct_full_run_1788151795/direct_runtime.csv",
    "model_id": "facebook/seamless-m4t-v2-large",
    "model_revision": "5f8cc790b19fc3f67a61c105133b20b34e3dcb76",
    "tokenizer_class": "SeamlessM4TTokenizerFast",
    "unk_token": "<unk>",
    "unk_token_id": 1,
    "total_rows": 4200,
    "direct_unk_rows": 508,
    "direct_unk_row_rate": 0.12095238095238095,
    "total_literal_unk_occurrences": 599,
    "reference_rows_tokenizing_to_unk": 3498,
    "direct_unk_rows_whose_reference_tokenizes_to_unk": 448,
    "repeated_sentence_pattern_counts": {
        "never_unk": 256,
        "always_unk": 43,
        "mixed": 2
    },
    "interpretation_note": "These are diagnostic associations. They do not prove
that a single factor causes SeamlessM4T to generate <unk>."
}

```

Saved to: /Users/nlh/Downloads/uoa_lectures/CS760/project/cs760-accent-st-

robustness-ml-research/runs/direct_unk_diagnostic_1789953533

1.13 Interpretation guide

- **References tokenize with UNK:** strong evidence of target-tokenizer coverage limitations.
- **References encode normally but generation still emits UNK:** more consistent with model-generation/decoding behaviour.
- **The same repeated sentence produces UNK across different speakers:** supports a content/model pattern more than a purely speaker-specific pattern.
- **Accent rates are similar:** do not present <unk> as an accent-specific result.

The official Direct outputs should remain unchanged.